

基於多智慧體系統之智慧交通導航與對話助理平台

Abstract	3
1. 系統架構設計	4
1.1. 系統架構圖	4
1.2. 技術棧	5
2. 資料建模與存儲	6
2.1. ER diagram	6
2.1.1. User & UserProfile	6
2.1.2. TimescaleDB Hyper-tables	7
3. API 與介面規範	8
3.1. 移動端 API	8
3.2. 外部對接表	9
4. 業務邏輯和流程分析	9
4.1. Sequence Diagram	9
4.1.1. User registration	9
4.1.2. User Login	10
4.1.3. Chat Manager 與 LLM 推論流程	11
4.1.4. 影像處理與資料儲存	11
5. 非功能性需求與系統安全 (Technical Constraints & Security)	12
5.1. 身份驗證與授權 (Authentication & Authorization)	12
5.2. 併發處理與效能 (Concurrency & Performance)	12
5.3. 監控與日誌 (Observability)	13
6. 結論	13

Abstract

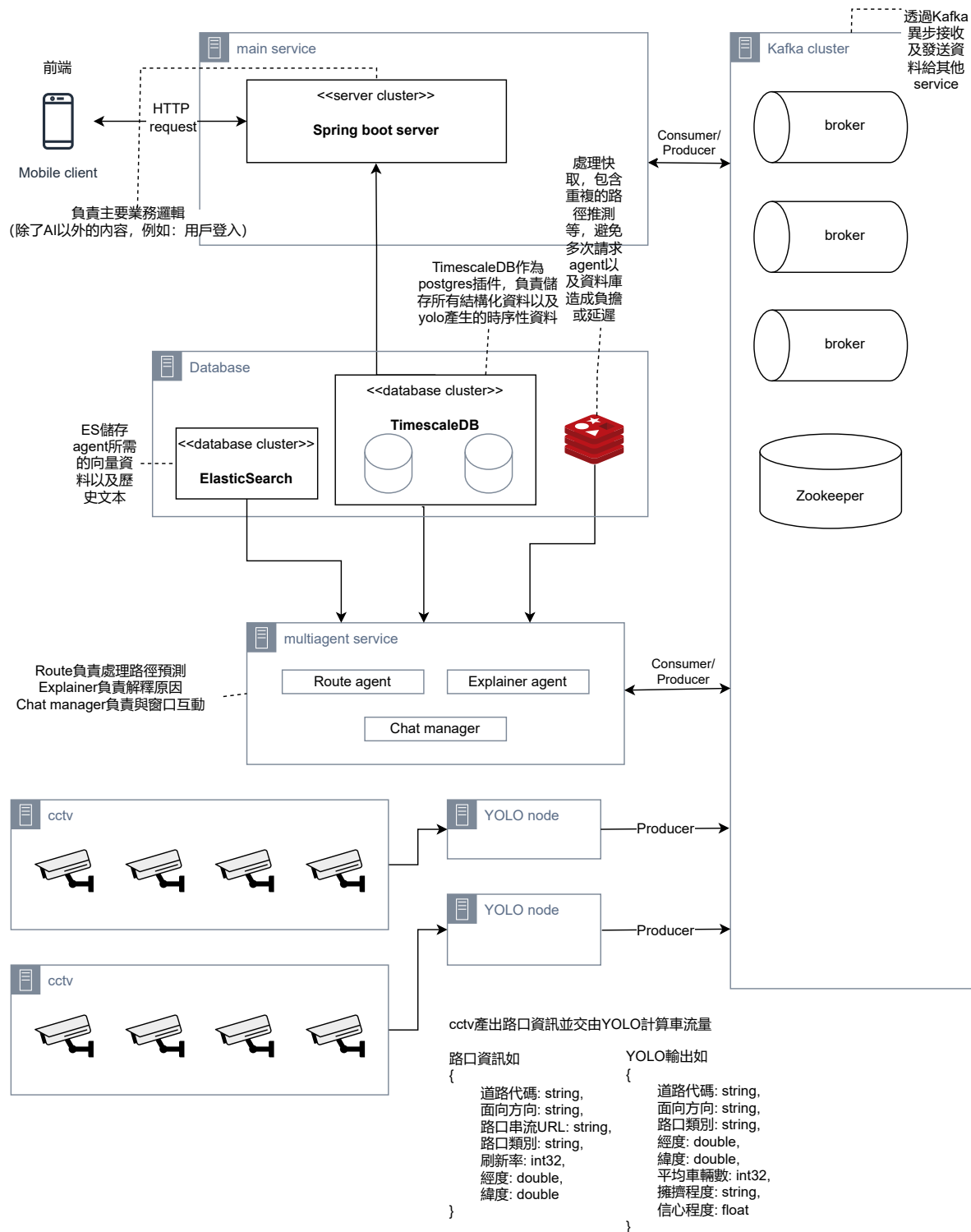
本專題旨在解決現代城市交通面臨的壅塞、停車位難尋及資訊分散整合困難等問題。我們設計並實作了一套結合**多智慧體系統 (Multi-Agent System, MAS)**、**微服務架構**與**即時數據串流技術**的智慧交通導航與對話助理平台。

系統核心採用 Agent 協作架構，透過 Group Chat Manager 協調 Route Agent、Parking Agent 及 Forecast Agent 等專家智慧體，並整合 **YOLO** 影像辨識、**LSTM/GRU** 流量預測及 **RAG (Retrieval-Augmented Generation)** 技術。使用者可透過自然語言與 Chatbot 互動，獲取考量即時流量與預測模型的動態路徑推薦，以及整合即時影像分析的停車位資訊。

技術實作上，本平台運用 **Kafka** 建立事件驅動架構，並以 **gRPC** 進行高效內部通訊，搭配 **TimescaleDB** 與 **Redis** 處理時序數據與快取，確保系統在大數據流量下的即時性與穩定性。本研究期能透過創新的軟體架構與 AI 整合，提供更直覺且智慧化的交通解決方案，有效提升駕駛人的出行效率。

1. 系統架構設計

1.1. 系統架構圖



1.2. 技術棧

application	Mobile
Service	Traffic prediction Transportation chat bot Path recommendation
API	HTTP call

AI

data process	python script opencv airflow
model	LSTM/GRU RAG YOLO

Server

Distributed System

Platform	AWS Local
Tech stack	Spring boot fastAPI docker k8s
monitor	OpenTelemetry Prometheus
Database	TimescaleDB Redis Elastic Search Kafka

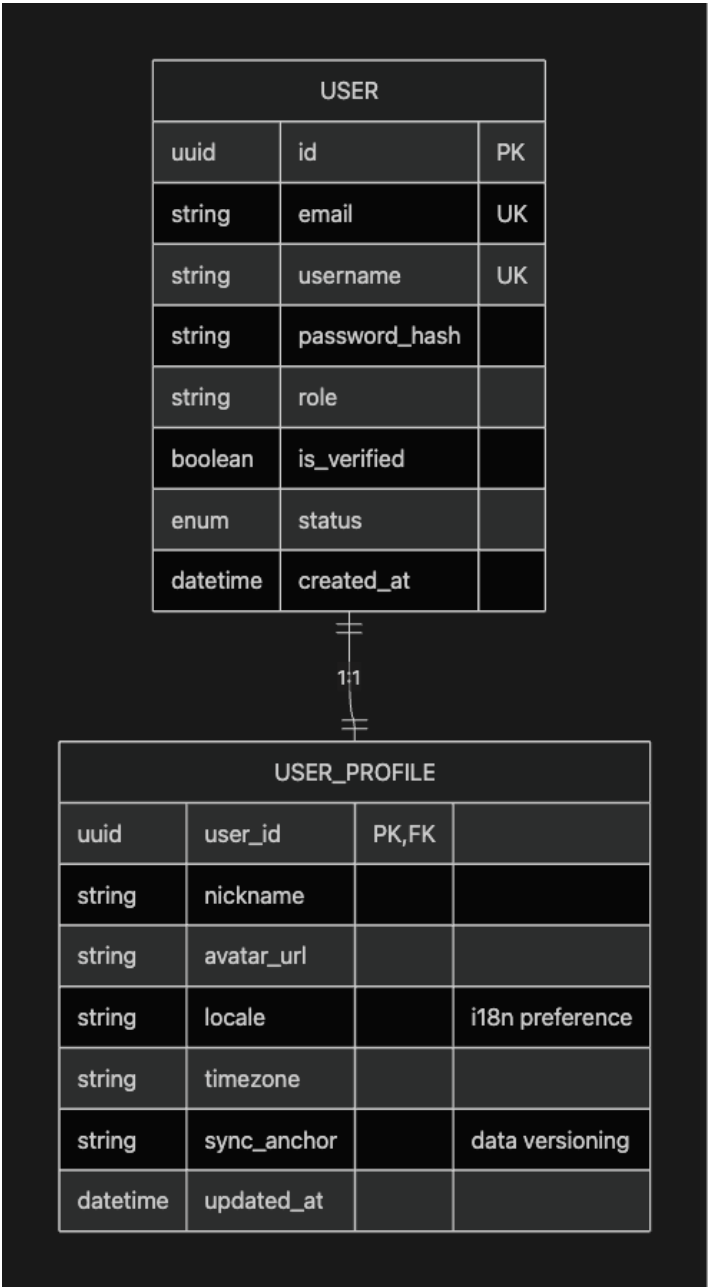
Client

Platform	Kotlin Multiplatform
----------	-------------------------

2. 資料建模與存儲

2.1. ER diagram

2.1.1. User & UserProfile



2.1.2. TimescaleDB Hyper-tables



3. API 與介面規範

3.1. 移動端 API

功能分類	請求方法	API 窗口 (Endpoint)	請求參數 (Request Body / Params)	回傳結果 (Response Body)	說明
身分驗證	POST	/api/v1/auth/register	{ "email": "string", "username": "string", "password_hash": "string" }	{ "id": "UUID", "message": "Success" }	用戶註冊
身分驗證	POST	/api/v1/auth/login	{ "email": "string", "password_hash": "string" }	{ "token": "JWT_STRING", "user": { "id": "UUID", "role": "string" } }	登入並取得授權
路況監控	GET	/api/v1/traffic/nodes	無	[{ "id": "UUID", "name": "string", "lat": 0.0, "lng": 0.0 }]	取得路口監視器清單
路況監控	GET	/api/v1/traffic/{id}/current	id: UUID (Path)	{ "congestion_score": 0.85, "level": "E", "vehicle_count": 45, "timestamp": "ISO-8601" }	獲取 YOLO 處理後的即時路況
智能推論	POST	/api/v1/route/recommend	{ "origin": "lat,lng", "destination": "lat,lng", "preferences": "string" }	{ "route_id": "UUID", "path": "GEOMETRY_STR", "estimated_time": "integer" }	請求 Route Agent 規劃路徑
智能推論	GET	/api/v1/route/{id}/explain	id: UUID (Path)	{ "reasoning": "string", "forecast_data": { "trend": "increasing" } }	結合 Explainer 與 Forecast Agent 的推論理由
智能對話	POST	/api/v1/chat/message	{ "session_id": "UUID", "content": "string" }	{ "reply": "string", "suggested_actions": ["list"] }	與 Chat Manager 對話
資料同步	GET	/api/v1/user/sync	last_sync_time: Long(Query)	{ "profile": { "nickname": "string", "avatar_url": "string" }, "is_updated": boolean }	Frontend 同步更新

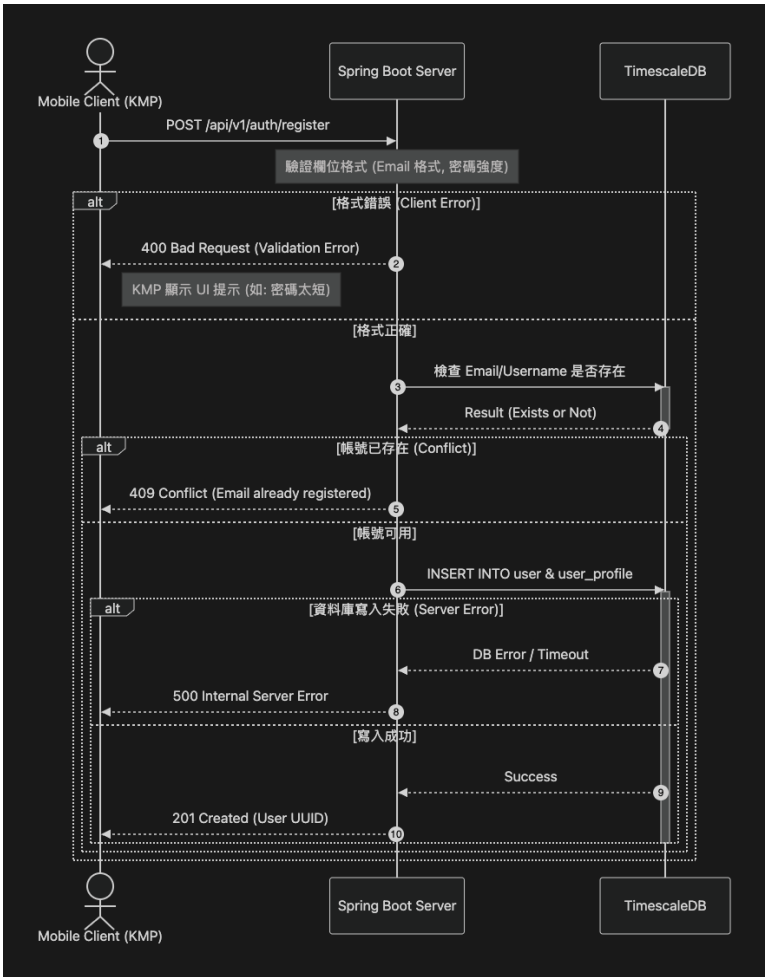
3.2. 外部對接表¹

數據對象	來源類型	傳輸協議	更新頻率	關鍵參數 (Fields)	內部處理邏輯
CCTV 串流/影像	TDX API	HTTPS	每 5~10 秒	camera_id, image_url, timestamp	由 YOLO Node 抓取影像 並進行物件偵測
路網靜 態資料	外部地圖服 務	REST API	每日 一次	intersection_name, coordinates, road_type	寫入 TRAFFIC_NODE 靜態表作為 LLM 推論基 礎

4. 業務邏輯和流程分析

4.1. Sequence Diagram

4.1.1. User registration ²

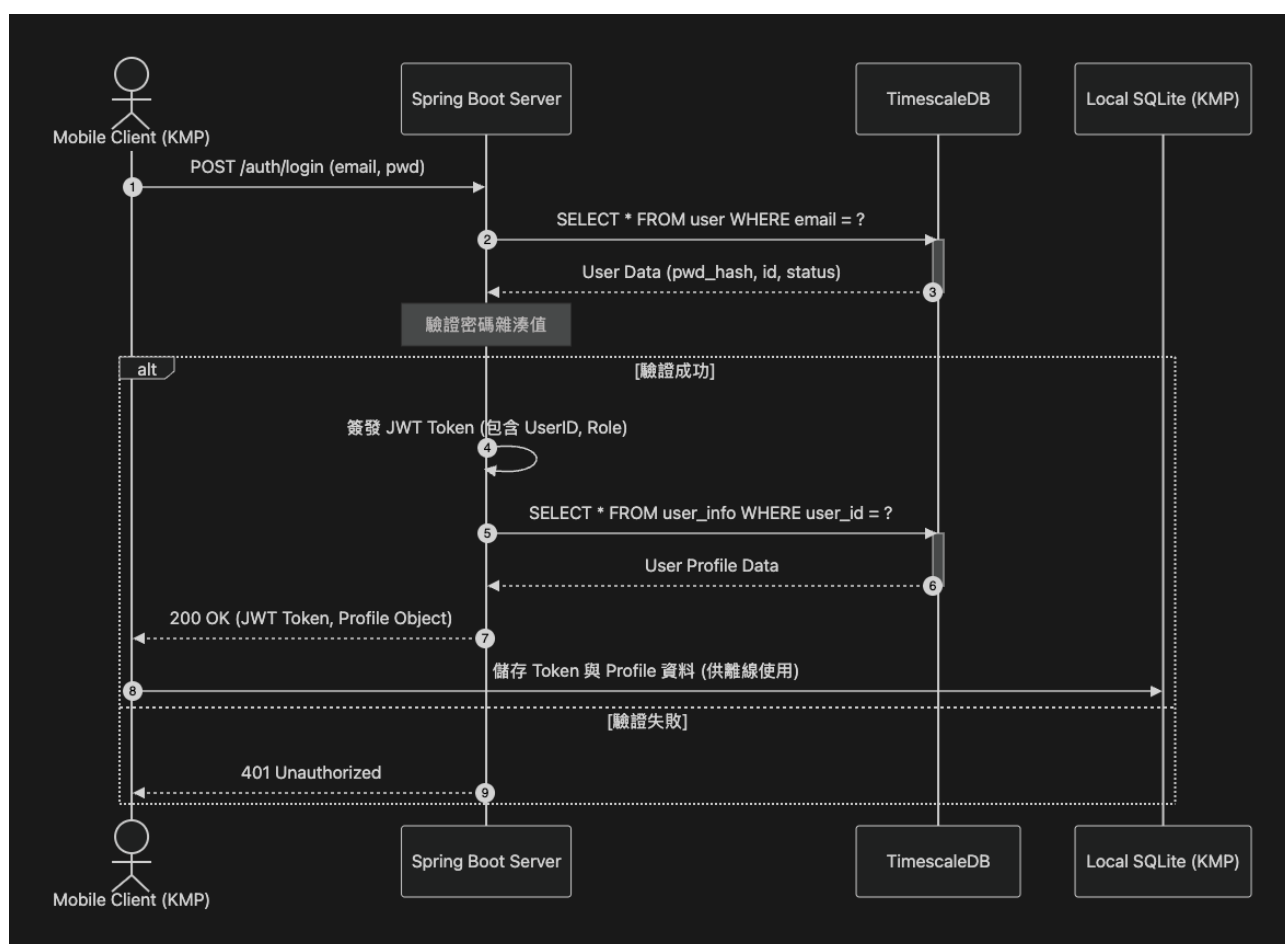


¹ TDX 資料平台不提供rtsp協議的cctv影像

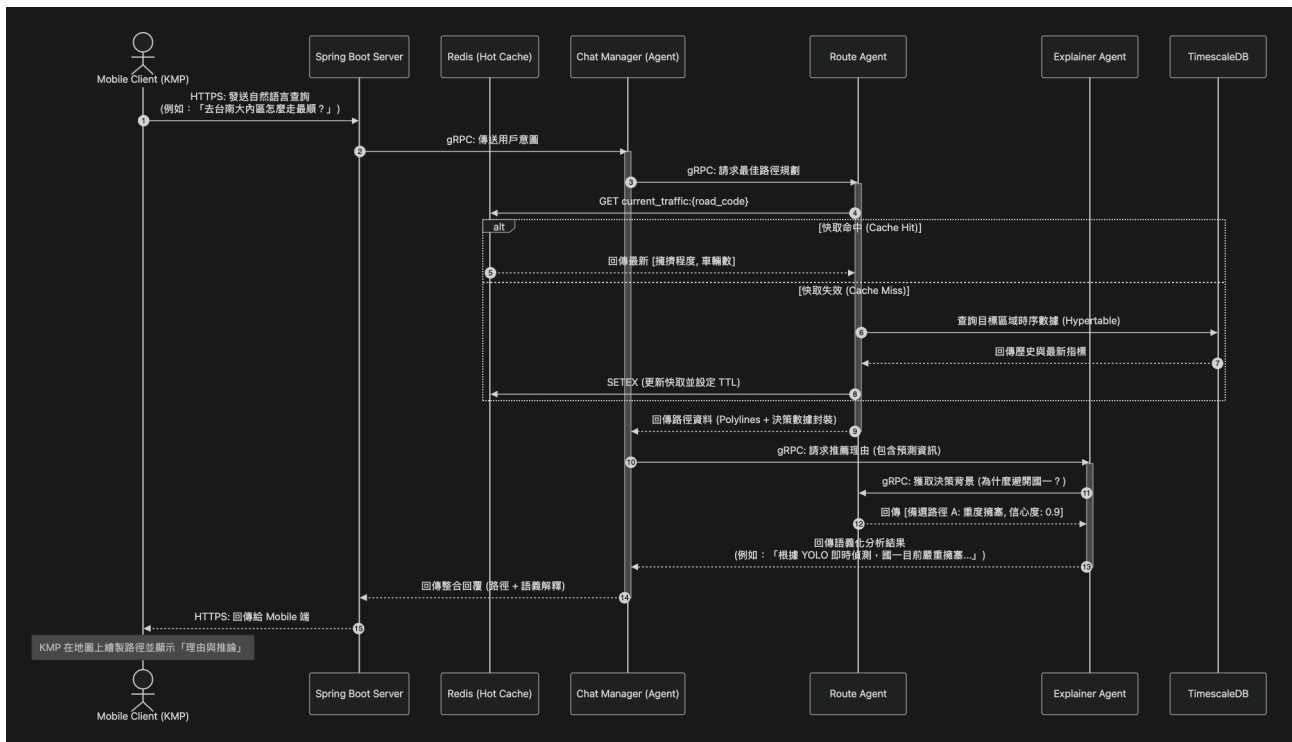
² HTTP status 補充在下表

HTTP 狀態碼	錯誤類型 (Error Type)	回傳 JSON 範例	前端處理建議 (Mobile Side)
400	INVALID_INPUT	{ "error": "Invalid email format" }	顯示紅色警告文字於輸入框下方
409	USER_ALREADY_EXISTS	{ "error": "Email is already taken" }	導向登入頁面或提示找回密码
429	TOO_MANY_REQUESTS	{ "error": "Rate limit exceeded" }	實施冷卻計時器，暫時停用註冊按鈕
500	DATABASE_ERROR	{ "error": "Service unavailable" }	顯示「伺服器維護中，請稍後再試」
503	NETWORK_TIMEOUT	{ "error": "Timed out" }	提示檢查網路連線或自動重試

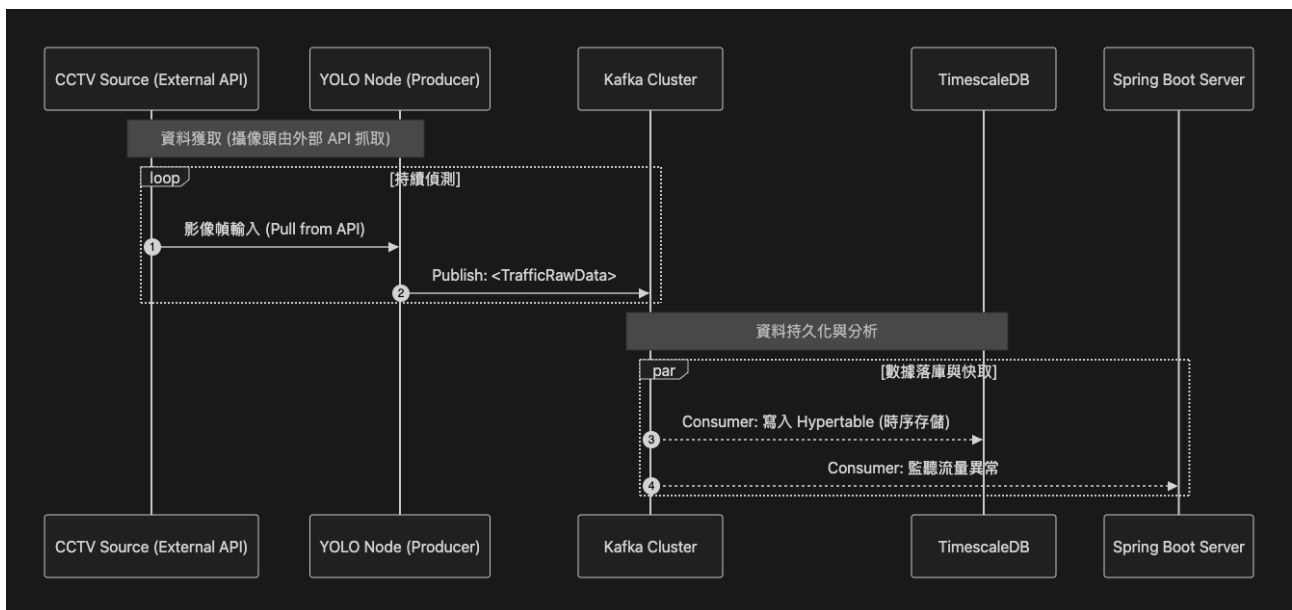
4.1.2. User Login



4.1.3. Chat Manager 與 LLM 推論流程



4.1.4. 影像處理與資料儲存



交互組件	協議 / 機制	目的
YOLO → Kafka	Kafka Producer	實現高吞吐量的影像偵測數據解耦，避免 YOLO 處理速度受資料庫寫入影響。

Kafka → TimescaleDB	TimescaleDB Sink	將時序數據持久化至 Hypertable ¹⁰ ，供 Forecast agent 進行模型訓練與回測。
Multi-Agent → gRPC	gRPC	用於 Route agent 與 Explainer agent 之間的同步請求，例如請求對特定擁塞路徑的「合理解釋」。
Kafka → Spring Boot	Consumer	讓後端主動得知擁塞狀態，以便向 Mobile client 發送 WebSocket 或 Push Notification。

5. 非功能性需求與系統安全 (Technical Constraints & Security)

本章節定義了系統在處理大量 YOLO 影像數據與多智能體推論時的安全性與穩定性標準。

5.1. 身份驗證與授權 (Authentication & Authorization)

系統採用 無狀態 (Stateless) 的安全架構，以支撐分散式叢集需求：

- **JWT (JSON Web Token)**：Mobile Client 登入後，Spring Boot 簽發包含 User UUID 與 Role 的 JWT。後續所有 HTTPS 與 gRPC 請求均需攜帶此 Token。
- **OAuth2 / OpenID Connect (預留)**：架構圖中 Spring Boot server 可作為 Resource Server，未來若對接外部 CCTV 廠商 API 或 Google 登入時，可快速擴展 OAuth2 流程。
- **傳輸安全**：所有 Mobile 端與伺服器間的通訊強制使用 TLS 1.3 (HTTPS) 加密；內部組件（如 Spring Boot 與 Agents 之間）使用帶有 SSL 的 gRPC。

5.2. 併發處理與效能 (Concurrency & Performance)

由於 YOLO 每秒產生大量事件，系統設計以下機制避免阻塞：

- **非同步訊息隊列 (Kafka)**：YOLO Node 與後端服務徹底解耦。當流量激增時，Kafka 作為緩衝區，由 Stream Processor 依據處理能力消費數據，確保系統不會因瞬間高負載崩潰。

- **Redis 分散式鎖 (Locking)**：在處理特定路口的 TrafficCongestionAlert 時，若多個 Agent 同時計算同一區域，利用 Redis 實作分散式鎖，確保針對單一事件的重算任務 (Re-routing) 不會重複執行。
- **TimescaleDB Hypertable**：利用時間分區特性，解決傳統 RDBMS 在大數據量下的索引失效問題，確保 LLM Agent 查詢歷史數據時維持毫秒級回應。

5.3. 監控與日誌 (Observability)

為確保 Multi-agent service 的自動化推論可追蹤：

- **分散式鏈路追蹤 (Distributed Tracing)**：導入 OpenTelemetry。當 User 發起路徑請求，系統會記錄從 Spring Boot -> Chat Manager -> Route Agent 的完整調用鏈路與耗時，便於定位推論延遲。
- **結構化日誌 (Centralized Logging)**：所有 Agent 與 YOLO Node 的日誌統一透過 Kafka 傳輸至 ELK/Loki 叢集。特別記錄 YOLO 偵測錯誤率與 Agent 推論失敗原因。
- **即時監控面板**：監控 Kafka 堆積量 (Lag) 與 TimescaleDB 的寫入吞吐量，當 YOLO 數據延遲超過 30 秒時自動觸發告警。

維度	技術實作	目的
安全性	JWT + bcrypt 雜湊	保護用戶隱私並確保請求合法性。
可擴展性	Kafka Broker 叢集	支援未來增加更多 YOLO 偵測節點。
低延遲	Redis 緩存	快速回應 Mobile 端對熱點路口狀態的查詢。
推論透明度	Explainer Agent	讓使用者了解 AI 推薦路徑的背後邏輯。

6. 結論

本專案為構建一個高度整合、可擴展且具備「認知推論」能力的智慧交通決策平台。透過 **Spring Boot** 作為核心閘道器，系統橋接了前端 **Kotlin Multiplatform** 的流暢體驗與後端 **YOLO/AI Agents** 的強大運算力。

本設計的關鍵價值在於：

- **時序數據的深度應用**：利用 **TimescaleDB** 的 Hypertable 技術，將 YOLO 產生的瞬時流量數據轉化為具備預測價值的時序資產，支撐 **Forecast Agent** 進行精準的路況預測。
- **非同步與解耦的穩定性**：藉由 **Kafka 叢集** 緩衝大規模的影像監控事件，確保系統在高負載路況下仍能維持優異的響應速度與數據完整性。
- **以人為本的智能推論**：透過 **Chat Manager** 整合 **Route** 與 **Explainer Agents**，將艱澀的交通數據轉化為直觀、具備解釋性的導航建議，顯著提升了終端用戶的信任度。

未來，隨著系統上線與數據積累，本架構可進一步導入更複雜的強化學習模型，實現自動化的交通號誌協同控制，為智慧城市（Smart City）的願景提供實質性的技術支撐。